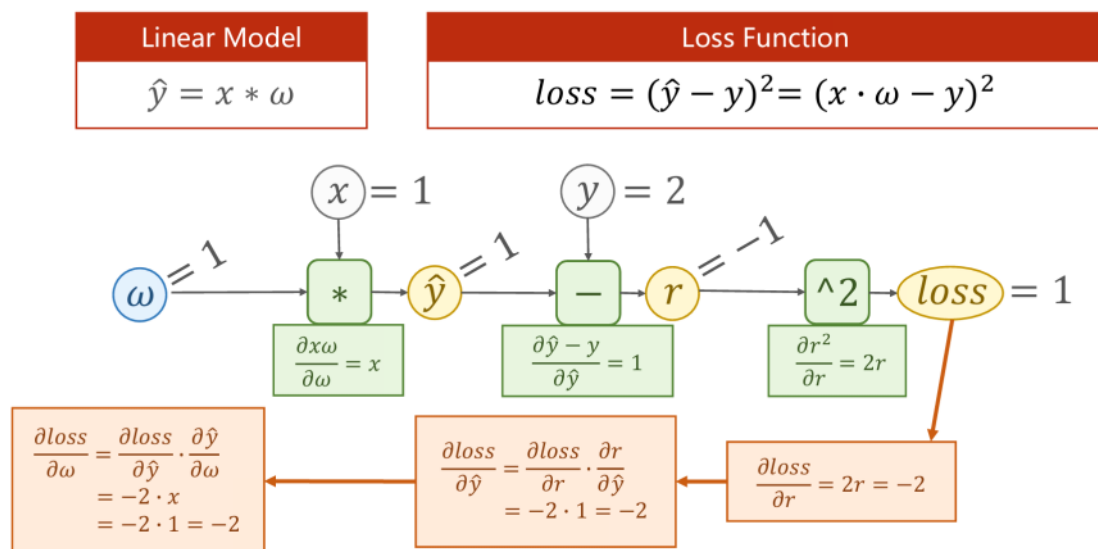


【pytorch04】反向传播

2026年9月7日 20:47

1. 动机：复杂（多层）网络无法手推解析梯度，需要**计算图** + **链式法则**自动求导。
2. 关键认知：多层线性层若不引入非线性激活函数，会退化成单一线性变换（需每层配非线性函数）。
3. 链式法则四步：
 1. 前向传播构建**计算图**；
 2. 计算每个节点的**局部梯度**；
 3. 接收后继节点传来的**梯度**；
 4. 用链式法则将梯度**反向回传**（Backward）。

Computational Graph of Linear Model



4. PyTorch 中的 Tensor：同时保存 **data**（节点值）和 **grad**（对 loss 的梯度）。
5. 自动求导机制（autograd）：
 1. `w.requires_grad = True`：开启对该张量的梯度追踪；
 2. `l.backward()`：反向传播，计算所有 `requires_grad=True` 的张量的梯度；
 3. `w.grad`：访问梯度。
6. 手动更新权重：`w.data = w.data - 0.01 * w.grad.data`（用 `.data` 绕开计算图）。
7. **Δ 梯度累加陷阱**：`.backward()` 计算的梯度会累加，更新后必须 `w.grad.data.zero_()` 清零。
8. `.item()`：把单元素张量转为 Python 标量（打印用）。

附：代码

```
import torch
```

```
x_data = [1.0, 2.0, 3.0]
```

```
y_data = [2.0, 4.0, 6.0]
```

```

w = torch.Tensor([1.0])          # 权重是一个 Tensor
w.requires_grad = True           # 需要计算梯度, 必须设为 True, 默认不计算梯度

# 定义模型 (线性模型  $y = w \cdot x$ )
def forward(x):
    return x * w

# 定义损失函数 (单样本)
def loss(x, y):
    y_pred = forward(x)
    return (y_pred - y) ** 2

print("predict (before training)", 4, forward(4).item())

for epoch in range(100):
    for x, y in zip(x_data, y_data):
        l = loss(x, y)           # 前向, 计算损失
        l.backward()             # 反向, 自动求梯度
        print('\tgrad:', x, y, w.grad.item()) # item取标量
        w.data = w.data - 0.01 * w.grad.data # 用梯度更新权重 (.data 绕开计算图)

    w.grad.data.zero_()          # 梯度会累加, 更新后必须清零!!

    print("progress:", epoch, l.item())

print("predict (after training)", 4, forward(4).item())

```